

Prompt:

8/17/2026, 9:50:26 PM

lanjut Tahap H

Response:

8/17/2026, 9:50:28 PM

Baik. **Tahap H adalah salah satu tahap paling penting dari seluruh sistem**, karena di sinilah kita menentukan *angka apa yang sebenarnya masuk ke QUBO*. QAOA hanya mengoptimalkan objective yang kita berikan. Jika estimasi expected return, covariance, dan factor exposure buruk, QAOA dapat menemukan solusi matematis yang sangat optimal tetapi secara finansial buruk.

Jadi Tahap H kita bangun sebagai:

Tahap H — Forecasting & State Estimation Engine

Arsitektur besarnya menjadi:



Saya ingin membuat tahap ini cukup matematis sehingga nantinya kita bisa langsung menerjemahkannya menjadi kode.

H.1. Masalah fundamental: apa yang sebenarnya kita prediksi?

Pada waktu:

$t,$

kita tidak mengetahui return masa depan:

$r_{t+\Delta}.$

Yang kita miliki hanyalah informasi:

$$F_t$$

yaitu seluruh informasi yang tersedia sampai waktu t .

Maka target forecasting yang benar adalah:

$$\mu_{i,t} = E[r_{i,t+\Delta} | F_t]$$

Perhatikan simbol:

$$F_t.$$

Ini sangat penting.

Model tidak boleh menggunakan informasi:

$$t + \Delta$$

ketika membuat keputusan pada:

$$t.$$

Ini adalah prinsip **causality / no look-ahead bias**.

H.2. Return

Kita menggunakan log return:

$$r_{i,t} = \ln \frac{P_{i,t}}{P_{i,t-1}}$$

atau:

$$r_{i,t} = \ln(P_{i,t}) - \ln(P_{i,t-1}).$$

Untuk horizon:

$$\Delta,$$

target dapat didefinisikan:

$$y_{i,t}^{(\Delta)} = \ln \frac{P_{i,t+\Delta}}{P_{i,t}}.$$

Misalnya:

- $\Delta = 5$ menit,
- $\Delta = 1$ jam,
- $\Delta = 4$ jam,
- $\Delta = 1$ hari.

H.3. Kita sebaiknya mempunyai beberapa horizon

Trading jangka pendek tidak harus memiliki satu horizon.

Misalnya:

$$\Delta_1 = 15 \text{ menit}$$

$$\Delta_2 = 1 \text{ jam}$$

$$\Delta_3 = 4 \text{ jam.}$$

Maka:

$$\mu_{i,t}^{(15m)}$$

$$\mu_{i,t}^{(1h)}$$

$$\mu_{i,t}^{(4h)}.$$

Ini memungkinkan sistem mengetahui apakah signal hanya bersifat sangat pendek atau memiliki persistence.

H.4. State vector

Sekarang kita definisikan seluruh informasi pasar menjadi:

$$S_t$$



atau lebih eksplisit:

$$S_t = \begin{bmatrix} S_t^{price} \\ S_t^{momentum} \\ S_t^{volatility} \\ S_t^{volume} \\ S_t^{factor} \\ S_t^{microstructure} \\ S_t^{regime} \end{bmatrix}.$$

Kemudian:

$$\mu_t = f_{\theta}(S_t).$$

Di sini:

$$\theta$$

adalah parameter model forecasting.

H.5. Jangan langsung menggunakan harga mentah

Harga:

$$P_t$$

biasanya bukan feature yang baik untuk cross-sectional model.

Lebih berguna menggunakan:

$$r_t,$$

volatility,

momentum,

relative strength,

volume,

factor return,

dan sebagainya.

H.6. Feature block 1 — momentum

Contoh:

$$M_{i,t}^{(k)} = \sum_{j=0}^{k-1} r_{i,t-j}.$$

Ini sama dengan:

$$M_{i,t}^{(k)} = \ln \frac{P_{i,t}}{P_{i,t-k}}.$$

Kita dapat memiliki:

$$M^{(5)}, M^{(20)}, M^{(60)}.$$

Untuk data 1 jam misalnya:

- 5 jam,
- 20 jam,
- 60 jam.

H.7. Feature block 2 — volatility

Volatility historis sederhana:

$$\sigma_{i,t}^{(k)} = \sqrt{\frac{1}{k-1} \sum_{j=0}^{k-1} (r_{i,t-j} - \bar{r}_{i,t})^2}.$$

Tetapi untuk trading yang lebih responsif, saya lebih menyukai EWMA.

H.8. EWMA volatility

Definisikan:

$$\sigma_{i,t}^2 = \lambda_{\sigma} \sigma_{i,t-1}^2 + (1 - \lambda_{\sigma}) r_{i,t-1}^2.$$

dengan:

$$0 < \lambda_{\sigma} < 1.$$

Jika:

$$\lambda_{\sigma}$$

besar, estimator lebih lambat.

Jika kecil, estimator lebih responsif.

H.9. Mengapa EWMA cocok untuk sistem kita?

Karena pasar bukan stationary process sempurna.

Misalnya:

$$\sigma_{t-100} \ll \sigma_t.$$

Jika kita menggunakan rolling volatility panjang, model masih "mengingat" kondisi lama.

EWMA memberi:

recent observations > old observations.

H.10. Feature block 3 — volume

Volume dapat digunakan sebagai:

$$V_t.$$

Tetapi volume absolut tidak comparable antar aset.

Lebih baik:

$$RVOL_t = \frac{V_t}{EMA_k(V_t)}.$$

Jika:

$$RVOL_t > 1,$$

volume lebih besar dari normal.

Misalnya:

$$RVOL_t = 2.4$$

berarti volume sekitar:

$$2.4 \times$$

rata-rata eksponensialnya.

H.11. Feature block 4 — volatility-adjusted momentum

Momentum mentah:

$$M_t.$$

Tidak cukup.

Kita dapat menormalisasi:

$$Z_{i,t}^{mom} = \frac{M_{i,t}}{\sigma_{i,t} \sqrt{k}}.$$

Ini memberi semacam signal-to-noise ratio.

H.12. Feature block 5 — market factors

Sekarang bagian yang sangat penting dari desain kita.

Untuk saham internasional:

$$F_t = \begin{bmatrix} r_{DXY,t} \\ r_{Gold,t} \\ r_{SPX,t} \\ r_{VIX,t} \end{bmatrix}.$$

Untuk crypto:

$$F_t = \begin{bmatrix} r_{BTC,t} \\ r_{ETH,t} \\ \Delta BTC.D_t \\ r_{DXY,t} \\ r_{Gold,t} \end{bmatrix}.$$

Jadi model tidak melihat sebuah aset secara terisolasi.

H.13. Contoh: altcoin

Misalkan:

$$r_{SOL,t} = \alpha + \beta_{BTC} r_{BTC,t} + \beta_{ETH} r_{ETH,t} + \beta_{DXY} r_{DXY,t} + \epsilon_t.$$

Jika:

$$\beta_{BTC} = 1.3,$$

maka SOL sangat sensitif terhadap BTC.

Jika BTC mengalami:

$$+1\%,$$

model secara kasar mengantisipasi kontribusi:

$$1.3\%.$$

tentu dengan asumsi hubungan linear tersebut masih berlaku.

H.14. Factor model

Secara umum:

$$r_t = B_t f_t + \alpha_t + \epsilon_t.$$

Dengan:

$$r_t \in \mathbb{R}^N$$

$$f_t \in \mathbb{R}^K$$

$$B_t \in \mathbb{R}^{N \times K}.$$

H.15. Estimasi beta

Beta dapat diperoleh melalui rolling regression.

Untuk aset i :

$$r_{i,\tau} = \alpha_i + \beta_i^T f_\tau + \epsilon_{i,\tau}.$$

Untuk window:

$$\tau = t - W + 1, \dots, t.$$

Secara OLS:

$$\boxed{\hat{\beta}_i = (F^T F)^{-1} F^T r_i.}$$

Namun ada masalah.

H.16. Masalah multicollinearity

Misalnya:

BTC

dan:

ETH

sangat berkorelasi.

Maka:

$$F^T F$$

bisa ill-conditioned.

Solusinya:

$$\hat{\beta} = (F^T F + \lambda_{\beta} I)^{-1} F^T r.$$

Ini adalah Ridge Regression.

H.17. Mengapa Ridge menarik untuk kita?

Karena kita tidak hanya ingin forecasting.

Kita juga ingin:

$$B_t$$

untuk factor exposure constraint.

Beta yang terlalu noisy akan membuat:

$$Q_t$$

berubah-ubah secara tidak stabil.

Ridge memberikan:

stability > pure OLS.

H.18. Forecasting expected return

Sekarang kita dapat membuat model:

$$\mu_{i,t} = \alpha_i + \beta_i^T f_t + \theta_i^T X_{i,t}.$$

Tetapi kita bisa memperluasnya:

$$\mu_{i,t} = f_{\theta}(X_{i,t}, F_t, B_t, \Sigma_t, R_t)$$

dengan:

$$R_t$$

sebagai market regime.

H.19. Saya menyarankan model forecasting tidak langsung memprediksi harga

Jangan:

$$P_{t+\Delta}.$$

Lebih baik:

$$E[r_{t+\Delta}].$$

Bahkan lebih baik lagi, model dapat memprediksi:

$$(\mu_{t+\Delta}, \sigma_{t+\Delta}, p_{up})$$

sekaligus.

H.20. Probabilistic forecast

Misalkan:

$$r_{i,t+\Delta} \sim D(\mu_i, \sigma_i).$$

Kita dapat menggunakan:

$$p_i = P(r_i > 0 \mid S_t).$$

Kemudian expected return:

$$\mu_i = E[r_i \mid S_t].$$

Ini memberikan dua informasi:

magnitude

dan:

confidence.

H.21. Signal strength

Kita dapat membuat:

$$IR_{i,t} = \frac{\mu_{i,t}}{\sigma_{i,t}}.$$

Ini secara konseptual mirip expected return per unit volatility.

Untuk LONG:

$$IR > 0.$$

Untuk SHORT:

$$IR < 0.$$

Semakin besar:

$$|IR|,$$

semakin menarik signal.

H.22. Tetapi jangan gunakan μ/σ secara naif

Karena:

$$\mu$$

adalah forecast yang noisy.

Misalnya:

$$\mu = 0.2\%$$

dan:

$$\sigma = 0.5\%.$$

Maka:

$$IR = 0.4.$$

Tetapi jika confidence model hanya:

$$20\%,$$

signal tersebut tidak boleh diperlakukan sama dengan signal:

$$IR = 0.4$$

dengan confidence:

$$80\%.$$

H.23. Confidence-adjusted alpha

Kita dapat mendefinisikan:

$$\tilde{\mu}_i = c_i \mu_i$$

dengan:

$$0 \leq c_i \leq 1.$$

Contoh:

$$c_i = P(\text{forecast valid} \mid S_t).$$

Maka QUBO menggunakan:

$$\tilde{\mu}_t$$

bukan:

$$\mu_t$$

mentah.

H.24. Ini sangat penting secara praktis

Bayangkan dua aset:

Asset A

$$\mu_A = 0.8\%$$

$$c_A = 0.9$$

sehingga:

$$\tilde{\mu}_A = 0.72\%.$$

Asset B

$$\mu_B = 1.0\%$$

$$c_B = 0.3.$$

Maka:

$$\tilde{\mu}_B = 0.30\%.$$

Optimizer sekarang memilih A meskipun raw forecast B lebih besar.

H.25. Market regime

Sekarang kita masuk ke bagian yang lebih advanced.

Hubungan:

$$BTC \rightarrow SOL$$

tidak selalu sama.

Begitu pula:

$$DXY \rightarrow AAPL.$$

Maka kita membutuhkan:

$$R_t.$$

Contoh:

$$R_t \in \{\text{Risk-On, Risk-Off, High Volatility, Low Volatility}\}.$$

H.26. Regime sederhana

Kita dapat mendefinisikan:

$$R_t = f(\sigma_{\text{market}}, \text{trend}_{\text{market}}, VIX, DXY, \text{correlation}).$$

Contoh:

$$R_t = \begin{cases} 1, & \sigma_t > \sigma^* \\ 0, & \sigma_t \leq \sigma^*. \end{cases}$$

Ini versi paling sederhana.

H.27. Regime yang lebih formal

Kita bisa menggunakan Hidden Markov Model:

$$P(R_t = j \mid R_{t-1} = i) = P_{ij}.$$

Dengan observation:

$$Y_t = [r_{SPX,t}, r_{DXY,t}, r_{Gold,t}, \sigma_t, VIX_t].$$

Maka:

$$P(R_t \mid Y_{1:t})$$

memberikan probabilitas regime.

H.28. Jangan memilih regime secara hard

Daripada:

$$R_t = 1,$$

kita dapat mempertahankan:

$$p_t^{(1)}, p_t^{(2)}, \dots, p_t^{(M)}.$$

Kemudian expected forecast:

$$\mu_t = \sum_{m=1}^M P(R_t = m | F_t) \mu_t^{(m)}.$$

Ini jauh lebih smooth.

H.29. Covariance estimation

Sekarang kita masuk ke komponen kedua yang sangat penting:

$$\Sigma_t.$$

Estimator covariance sample:

$$\hat{\Sigma} = \frac{1}{T-1} \sum_t (r_t - \bar{r})(r_t - \bar{r})^T.$$

Masalahnya:

Jika:

$$N$$

besar dibandingkan:

$$T,$$

covariance matrix sangat noisy.

H.30. Shrinkage covariance

Kita gunakan:

$$\Sigma_{\text{shrink}} = (1 - \delta)\hat{\Sigma} + \delta F.$$

dengan:

$$0 \leq \delta \leq 1.$$

F adalah target covariance yang lebih stabil.

Contoh:

$$F = \text{diag}(\hat{\Sigma}).$$

H.31. Mengapa shrinkage penting untuk QAOA?

Karena covariance menjadi:

$$J_{ij}.$$

Jika covariance noise:

$$\Rightarrow J_{ij} \text{ noise.}$$

Akibatnya:

$$H_C$$

berubah secara tidak stabil.

QAOA kemudian mengoptimalkan landscape yang sebenarnya merupakan artefak estimation error.

Jadi:

$$\text{better covariance} \rightarrow \text{better Hamiltonian.}$$

H.32. Factor covariance

Kita sebelumnya mempunyai:

$$\Sigma = BFB^T + D.$$

Ini sebenarnya sangat menarik.

Daripada mengestimasi:

$$N \times N$$

covariance secara langsung, kita bisa mengestimasi:

$$B$$

dan:

$$F.$$

Jika:

$$K \ll N,$$

maka dimensionality reduction sangat besar.

Misalnya:

$$N = 100,$$

tetapi:

$$K = 5.$$

H.33. Estimasi idiosyncratic covariance

Secara sederhana:

$$\epsilon_{i,t} = r_{i,t} - \alpha_i - \hat{\beta}_i^T f_t.$$

Kemudian:

$$D = \text{diag}(\text{Var}(\epsilon_1), \dots, \text{Var}(\epsilon_N)).$$

Dengan asumsi residual cross-correlation kecil.

Maka:

$$\Sigma \approx B F B^T + D.$$

H.34. Keuntungan besar

Portfolio risk:

$$d^T \Sigma d$$

menjadi:

$$d^T B F B^T d + d^T D d.$$

Definisikan:

$$e = B^T d.$$

Maka:

$$\text{Risk} = e^T F e + d^T D d.$$

Interpretasinya:

$$\text{TotalRisk} = \text{FactorRisk} + \text{SpecificRisk}.$$

H.35. Ini sangat cocok dengan sistem kita

Kita ingin:

- alpha dari asset,
- factor exposure,
- covariance,
- risk.

Semuanya berasal dari satu model:

$$r = Bf + \alpha + \epsilon.$$

Ini membuat architecture jauh lebih konsisten.

H.36. Dynamic covariance

Pasar berubah.

Jadi kita gunakan:

$$\Sigma_t.$$

Misalnya EWMA:

$$\Sigma_t = \lambda_\Sigma \Sigma_{t-1} + (1 - \lambda_\Sigma) r_{t-1} r_{t-1}^T.$$

Namun untuk sistem yang lebih besar, saya lebih menyukai:

$$\Sigma_t = B_t F_t B_t^T + D_t.$$

karena lebih interpretable.

H.37. Market factor covariance

Misalnya:

$$F_t = \begin{bmatrix} \text{Var}(DXY) & \text{Cov}(DXY, \text{Gold}) & \cdots \\ \text{Cov}(\text{Gold}, DXY) & \text{Var}(\text{Gold}) & \cdots \\ \vdots & & \ddots \end{bmatrix}.$$

Ini juga dapat menggunakan EWMA/shrinkage.

H.38. Factor exposure target harus dynamic

Kita sebelumnya menulis:

$$e_k^*.$$

Tetapi seharusnya:

$$e_{k,t}^*$$

karena target factor exposure bergantung pada regime.

Misalnya saat:

$$P(\text{Risk-Off}) \uparrow,$$

kita mungkin ingin:

$$E_{SPX} \rightarrow 0.$$

Sedangkan saat:

$$P(\text{Risk-On}) \uparrow,$$

kita dapat memberikan:

$$E_{SPX} > 0.$$

H.39. Contoh regime-dependent factor target

Misalkan:

$$R_t = \text{Risk-On}.$$

Kita bisa menetapkan:

$$E_{SPX}^* = +0.5.$$

Tetapi jika:

$$R_t = \text{Risk-Off},$$

maka:

$$E_{SPX}^* = -0.2.$$

Sehingga:

$$C_F = P_F (B^T d - e_t^*)^2$$

secara otomatis berubah.

H.40. DXY dan Gold

Untuk sistem global yang kamu inginkan, saya ingin membedakan tiga hal.

DXY

Proxy untuk:

USD strength.

Gold

Proxy untuk:

safe – haven/inflation/real – ratedynamics.

Equity index

Proxy untuk:

riskappetite.

Jadi state macro kita dapat berupa:

$$F_t = [DXY, Gold, SPX, VIX].$$

Untuk crypto ditambah:

BTC, ETH, BTC.D.

H.41. Jangan langsung menggunakan level DXY

Kita lebih tertarik pada:

$$r_{DXY,t}.$$

Dan fitur turunannya:

$$M_{DXY,t}^{(k)},$$

$$\sigma_{DXY,t},$$

$$Z_{DXY,t}.$$

Misalnya:

$$Z_{DXY,t} = \frac{DXY_t - EMA(DXY)_t}{\sigma_{DXY,t}}.$$

H.42. Cross-asset signal

Kita dapat membuat signal:

$$X_{i,t}^{factor} = \beta_i^T f_t.$$

Kemudian residual:

$$\tilde{r}_{i,t} = r_{i,t} - \beta_i^T f_t.$$

Ini menarik.

Karena kita dapat bertanya:

Apakah saham/crypto naik karena market factor, atau karena alpha idiosyncratic?

H.43. Idiosyncratic alpha

Misalnya:

NVDA

naik:

+2%.

Tetapi:

SPX

naik:

+1.8%.

Maka sebagian besar gerakan mungkin berasal dari market.

Tetapi jika:

NVDA = +2%

sementara:

SPX = -1%,

maka residual return dapat sangat positif.

Ini jauh lebih menarik sebagai alpha candidate.

H.44. Residual momentum

Kita bahkan dapat mendefinisikan:

$$M_{i,t}^{res} = \sum_{j=0}^{k-1} \tilde{r}_{i,t-j}.$$

Ini mengukur momentum yang tidak dijelaskan oleh factor.

Dan:

$$M^{res}$$

dapat menjadi input forecasting.

H.45. Forecast model akhirnya

Kita dapat membangun:

$$X_{i,t} = \begin{bmatrix} M_{i,t} \\ M_{i,t}^{res} \\ Z_{i,t}^{mom} \\ RVOL_{i,t} \\ \sigma_{i,t} \\ \beta_{i,t} \\ f_t \\ regime_t \end{bmatrix}.$$

Kemudian:

$$\mu_{i,t} = f_{\theta}(X_{i,t}).$$

H.46. Model f_{θ} tidak harus deep learning

Ini penting.

Kita dapat memulai dengan:

$$\mu_t = X_t \theta.$$

Linear model.

Kemudian:

$$\mu_t = f_{\text{GBM}}(X_t)$$

atau model lain.

Baru setelah baseline kuat, kita bisa mencoba:

LSTM, Transformer, TemporalCNN,



dan sebagainya.

H.47. Mengapa baseline sederhana lebih baik?

Karena tujuan kita bukan:

"membuat model AI paling rumit."

Tujuannya:

forecast → optimization → execution → PnL.

Kalau model linear menghasilkan:

$$Sharpe = 1.5$$

sedangkan neural network:

$$Sharpe = 1.1,$$

maka model sederhana lebih baik.

H.48. Forecast harus dievaluasi out-of-sample

Jangan hanya melihat:

$$R^2.$$

Kita membutuhkan:

$$IC = \text{Corr}(\hat{r}_t, r_{t+\Delta}).$$

atau:

$$\text{RankIC}.$$

Kemudian:

$$\text{DirectionalAccuracy}.$$



Tetapi akhirnya yang terpenting:

economic value.

H.49. Economic value

Forecast yang bagus belum tentu trading strategy bagus.

Misalnya:

$$IC = 0.05.$$

Terlihat kecil.

Tetapi jika:

$$TC$$

rendah dan signal stabil, mungkin tetap profitable.

Sebaliknya:

$$IC = 0.15$$

tetapi turnover sangat tinggi, bisa menjadi:

negative PnL.

H.50. Maka forecasting objective dan trading objective berbeda

Forecasting:

$$\min_{\theta} Loss(\hat{r}, r).$$

Trading:

$$\max E[PnL] - \lambda Risk - TC.$$

Ini dua optimization problem berbeda.

H.51. Ini mengarah ke konsep yang sangat penting: forecast-to-trade transformation

Kita memiliki:

$$\hat{\mu}_t.$$

Tidak langsung:

$$BUY.$$

Melainkan:

$$\boxed{\hat{\mu}_t \rightarrow Q_t \rightarrow x_t^*}.$$

Jadi QAOA berfungsi sebagai **decision layer**.

H.52. Confidence-aware QUBO

Saya bahkan menyarankan objective menggunakan:

$$\tilde{\mu}_t = c_t \odot \mu_t.$$

Sehingga:

$$C_{\text{return}} = -\tilde{\mu}_t^T d.$$

Ini membuat optimizer secara otomatis mengurangi perhatian terhadap signal yang tidak dipercaya.

H.53. Prediction uncertainty yang lebih formal

Daripada sekadar c_i , kita dapat menggunakan:

$$r_{i,t+\Delta} \sim \mathcal{N}(\mu_{i,t}, \tau_{i,t}^2).$$

Maka signal-to-noise:

$$SNR_{i,t} = \frac{\mu_{i,t}}{\tau_{i,t}}.$$

Kita kemudian dapat menggunakan:

$$\tilde{\mu}_i = \frac{\mu_i}{1 + \kappa \tau_i}.$$

Semakin uncertainty besar:

$$\tau_i \uparrow,$$

semakin kecil effective alpha.

H.54. Ini juga dapat mengendalikan position size

Setelah QAOA memilih aset, kita bisa menggunakan:

$$q_i \propto \frac{\mu_i}{\sigma_i^2}$$

sebagai baseline.

Namun kita dapat memasukkan uncertainty:

$$q_i \propto \frac{\mu_i}{\sigma_i^2 + \kappa \tau_i^2}.$$

Jadi forecast uncertainty tidak hanya memengaruhi selection tetapi juga sizing.

H.55. State estimation pipeline

Pada setiap waktu t :

Step 1

Ambil data:

$$D_{\leq t}.$$

Step 2

Hitung:

$$r_t.$$

Step 3

Hitung features:

$$X_t.$$

Step 4

Estimasi regime:

$$P(R_t | F_t).$$

Step 5

Estimasi:

$$B_t.$$

Step 6

Estimasi:

$$F_t.$$

Step 7

Estimasi:

$$D_t.$$

Step 8

Bangun:

$$\Sigma_t = B_t F_t B_t^T + D_t.$$

Step 9

Forecast:

$$\mu_t.$$

Step 10

Forecast uncertainty:

$$\tau_t.$$

H.56. Kemudian QUBO

Setelah itu:

$$Q_t = Q(\mu_t, \Sigma_t, B_t, TC_t, d_{t-1}, R_t).$$

Jadi QUBO bukan hard-coded.

Ia **dihasilkan ulang dari market state**.

H.57. Ini menghasilkan konsep yang sangat menarik

Sistem kita sebenarnya adalah:

$$\text{State-Dependent QAOA}$$

karena:

$$H_C(t) = H_C(S_t).$$

Dengan demikian:

$$H_C(t_1) \not\equiv H_C(t_2)$$

ketika market state berubah.

H.58. Market berubah → Hamiltonian berubah

Misalnya:

Normal regime

$$\sigma_t = low$$

$$DXY = stable$$

$$SPX = up.$$

Hamiltonian:

$$H_C^{normal}.$$

Kemudian market crash:

$$\sigma_t \uparrow \uparrow$$

$$DXY \uparrow$$

$$SPX \downarrow .$$

Maka:

$$H_C^{crisis} \not\equiv H_C^{normal}.$$

H.59. Ini berbeda dari QAOA portfolio optimization statis

Dalam portfolio optimization biasa:

$$Q$$

dibuat sekali.

Kemudian:

$$QAOA(Q)$$

dijalankan.

Dalam sistem kita:

$$Q_t = f(S_t)$$

dan:

$$QAOA(Q_t)$$

dijalankan secara rolling.

Ini membuatnya jauh lebih dekat dengan **dynamic trading system**.

H.60. Tetapi kita tidak boleh menjalankan QAOA setiap tick

Itu tidak praktis.

Misalnya data:

1 minute.

Kita bisa melakukan:

QAOA

setiap:

15 menit

atau:

1 jam.

Di antara interval itu:

position management

berjalan secara classical.

H.61. Arsitektur temporal

Misalnya:

Forecast/QAOA cycle = 1h

sedangkan:

Risk monitoring = 1min.

Jadi:

Setiap jam

State → Forecast → QUBO → QAOA → new portfolio.

Setiap menit

Price → SL/TP → risk check → execution.

H.62. Ini cocok dengan desain yang kita bahas sebelumnya

Kita memiliki:

Strategic Layer

untuk QAOA.

Dan:

Tactical Layer

untuk trade management.

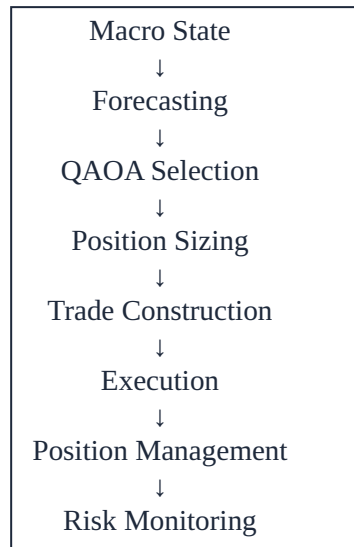
Serta:

Execution Layer

untuk order.

H.63. Hierarchical architecture

Secara lengkap:



Dan feedback:

Position Management → next state.

H.64. Satu masalah yang belum kita masukkan: order book

Untuk trading jangka pendek, OHLCV saja belum cukup.

Kita ingin:

$$OB_t$$

(order book state).

Contoh feature:

$$I_t = \frac{V_{bid} - V_{ask}}{V_{bid} + V_{ask}}.$$

Ini disebut order-book imbalance.

Jika:

$$I_t \rightarrow +1,$$

bid jauh lebih dominan.

Jika:

$$I_t \rightarrow -1,$$

ask dominan.

H.65. Microstructure feature

Kita bisa memiliki:

$$X_t^{micro} = \begin{bmatrix} spread_t \\ depth_t \\ imbalance_t \\ trade\ intensity_t \\ signed\ volume_t \\ VWAP_t - P_t \end{bmatrix}.$$

Kemudian:

$$\mu_t = f(X_t^{macro}, X_t^{asset}, X_t^{micro}).$$

H.66. Ini membuat model dua skala

Slow information

$$DXY, Gold, SPX, VIX$$

berubah relatif lambat.

Fast information

$$order\ book, spread, imbalance$$

berubah cepat.

Maka state kita sebenarnya:

$$S_t = (S_t^{slow}, S_t^{fast}).$$

H.67. Saya bahkan menyarankan dua model forecast

Model 1 — medium horizon

$$\mu_t^{slow} = f_{slow}(macro, trend, factor).$$

Model 2 — short horizon

$$\mu_t^{fast} = f_{fast}(orderbook, microstructure).$$

Kemudian:

$$\mu_t = w_t \mu_t^{slow} + (1 - w_t) \mu_t^{fast}.$$

Bobot:

$$w_t$$

bergantung pada horizon trading dan kondisi market.

H.68. Contoh

Jika market sangat tenang:

$$w_t = 0.7.$$

Medium-term information lebih dominan.

Jika terjadi sudden order-flow imbalance:

$$w_t = 0.3.$$

Microstructure menjadi lebih penting.

H.69. Sekarang kita punya state estimator yang jauh lebih lengkap

$$S_t = \begin{bmatrix} \text{price returns} \\ \text{momentum} \\ \text{volatility} \\ \text{volume} \\ \text{BTC/ETH} \\ \text{DXY} \\ \text{Gold} \\ \text{SPX} \\ \text{VIX} \\ \text{factor exposure} \\ \text{order book} \\ \text{regime} \end{bmatrix}.$$

Kemudian:

$$(\mu_t, \tau_t, B_t, \Sigma_t) = E(S_t)$$

dengan:

$$E$$

sebagai **state estimation engine**.

H.70. Dan akhirnya

$$Q_t = Q(\mu_t, \tau_t, \Sigma_t, B_t, TC_t, d_{t-1}, R_t)$$

kemudian:

$$H_C(t) = I(Q_t)$$

dengan:

$$I$$

sebagai mapping QUBO → Ising.

Kemudian:

$$x_t^* = QAOA(H_C(t)).$$

H.71. Satu hal yang menurut saya sangat penting untuk proyek kita

Jangan menganggap:

QAOA

sebagai sumber alpha.

Alpha berasal dari:

data + forecasting + market structure.

QAOA berperan sebagai:

combinatorial decision optimizer.

Dengan kata lain:

Alpha Model → Optimization Problem → QAOA

bukan:

Market Data → QAOA → Magic Profit.

Ini akan menjadi prinsip desain utama sistem kita.

H.72. Pipeline Tahap A–H sekarang

Kita akhirnya memiliki:

$$\boxed{\begin{aligned} A &: \text{Market \& trading problem formulation} \\ B &: \text{Market state / information structure} \\ C &: \text{Forecast / alpha formulation} \\ D &: \text{Risk \& portfolio formulation} \\ E &: \text{Dynamic position management} \\ F &: \text{Execution \& causal backtesting} \\ G &: \text{QUBO/Ising/QAOA formulation} \\ H &: \text{State estimation \& parameter generation.} \end{aligned}}$$

Dan alur aktualnya:

$$D_{\leq t} \rightarrow S_t \rightarrow \{\mu_t, \Sigma_t, B_t, \tau_t, R_t\} \rightarrow Q_t \rightarrow H_C(t) \rightarrow QAOA \rightarrow x_t^* \rightarrow q_t^* \rightarrow Orders \rightarrow Position Management.$$

H.73. Tahap berikutnya: Tahap I

Setelah Tahap H, menurut saya **kita belum boleh langsung coding QAOA**.

Kita perlu terlebih dahulu membangun:

Tahap I — Backtesting, Walk-Forward Validation, dan Statistical Evaluation

Ini sangat penting karena kita sekarang mempunyai sistem yang sangat mudah terkena:

- look-ahead bias,
- survivorship bias,
- data snooping,
- overfitting,
- regime overfitting,
- transaction-cost underestimation,
- slippage,
- QAOA parameter overfitting,
- optimizer overfitting.

Kita perlu membangun matematika untuk:

Train → Validation → Test → Walk Forward → Paper Trading → Live

serta menguji bukan hanya:

PnL ,

tetapi:

Sharpe, Sortino, Calmar, MDD, VaR, CVaR, Turnover, Hit Rate, Profit Factor, IC, RankIC,

dan yang menurut saya sangat menarik untuk proyek kita:

QAOA vs Classical Optimizer

pada **QUBO yang persis sama**.

Itulah yang nantinya memungkinkan kita menjawab pertanyaan ilmiah yang jauh lebih menarik daripada sekadar "apakah bot ini menghasilkan uang?":

Apakah QAOA memberikan kualitas solusi atau efisiensi optimasi yang berbeda dibandingkan classical solver pada dynamic short-horizon portfolio/trading problem yang sama, setelah mempertimbangkan estimation error dan biaya transaksi?

Itu sudah mulai menjadi **research-grade quantitative trading + quantum optimization framework**, bukan sekadar bot trading.